Failure (watch it loop, then measure the cost) — Hints

> Back to the exercise · Hint 1 · Hint 2 · Solution

Only 2 hints — work through them in order, and don't jump ahead until you've genuinely tried. Each hint has 2 depth levels: **Basic** (the plain idea) and **Intermediate** (proper Python). Read Basic first even if you already know Python — it's the fastest way to spot exactly what Intermediate adds.

- Hint 1 — A loop that never ends, safely
- Hint 2 — The plan, and almost the whole thing

Back to the exercise · Hint 1 · Hint 2 · Solution

Hint 1 — A loop that never ends, safely

Basic Version

You need a tool that always asks to be called again — `always_ask_again(city)`, whose description says "Always call it again to check". The model keeps calling it, so the loop only stops because of a limit.

Never run it without a limit. Set a huge step limit (`1000`) **and** a deadline: `deadline = time.time() + 10`, checked at the top of every turn — Doc02's "never a call without a cap".

> Back to the exercise · Hint 1 · Hint 2 · Solution

Intermediate Version

Put both limits in one function, `run_with_limits(task, max_iterations, timeout_seconds)`, and raise `MaxIterationsExceeded` (from the Intermediate exercise) for either one — steps or deadline.

Then measure the cost: each response has `response.usage_metadata["total_tokens"]` (LangChain's name for Doc04's `usage`). Store it in each step, add the steps up after the run stops, and compare with one direct call on an easy question.

Difference between Basic and Intermediate: Basic watches a loop run away and get stopped. Intermediate stops it with both limits in one function, and puts a number on what the looping cost.

Back to the exercise · Hint 1 · Hint 2 · Solution

Hint 2 — The plan, and almost the whole thing

Basic Version

```
max_iterations = 1000
deadline = time.time() + 10
for step in range(max_iterations):
    if time.time() > deadline: print "stopped by the deadline", stop
    think / act / observe, printing each step
```

> Back to the exercise · Hint 1 · Hint 2 · Solution

Intermediate Version

```
run_with_limits(task, max_iterations, timeout_seconds) -> str:
    for step ...:
        past the deadline? raise MaxIterationsExceeded("deadline passed", steps)
        think; done? return the answer
        act / observe; steps.append({..., "tokens": total_tokens})
    raise MaxIterationsExceeded(message, steps)

main: catch it, add up the tokens in e.steps; then one direct call's tokens
```

Write it, then compare against the Solution.

Difference between Basic and Intermediate: Basic is the watch-it-run-away script. Intermediate is the reusable, two-limit function plus a cost comparison.

Back to the exercise · Hint 1 · Hint 2 · Solution

Full solution: Show me the solution